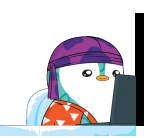
Linux Fundamentals



File Systems, Users & Permissions, Process & Logs, Installing & Removing Packages



Operating Systems

In Technical terms, An **operating system (OS)** is a software that acts as an intermediary between the computer's hardware and the user, managing all of the computer's resources like memory, processing, and devices.

Simply put, An Operating System is the bridge between the software and the hardware.

- Controls the computer’s hardware
- Allows you to run programs
- Makes everything work together smoothly

Extra learning material:
https://medium.com/@thecloudfairy/linux-management-for-cloud-engineering-part-1-399a6ca2c466

Imagine your computer is like a big office building:

- The hardware (CPU, memory, keyboard, screen, etc.) are like the tables, chairs, and rooms.
- The software programs (like Chrome, Word, VS Code) are like workers doing tasks.

But who manages the building?

👉 **That is the Operating System (OS).**

How Does the Concept of OSes Apply to Cloud Engineering?

In cloud engineering, you work with servers, virtual machines, containers, and services running on remote infrastructure (AWS, Azure, GCP, etc.). All these still rely on Operating Systems.

How do OSes apply in cloud engineering?

- **Servers run on OSes:** Every EC2 instance (AWS), VM (Azure), Compute Engine (GCP) is a virtual server that needs an OS. You choose whether it runs Linux (Ubuntu, Amazon Linux, CentOS) or Windows Server.
- **Deployment and configuration:** When you deploy applications in the cloud, they get installed on an OS. You write scripts (Bash for Linux, PowerShell for Windows) to automate setup.
- **Containers and OS-level virtualisation:** Docker containers share the host OS kernel. Kubernetes schedules containers onto nodes with OSes underneath.
- **OS knowledge for security and patching:** Cloud engineers manage OS updates, security patches, user permissions, and firewalls.
- **Understanding OS-level processes helps in troubleshooting and hardening.**
- **Cloud-native services still depend on OSes behind the scenes:** Even managed services (e.g. RDS databases, Lambda functions) run on underlying OSes maintained by the cloud provider.
- **Infrastructure as Code (IaC):** When writing Terraform or CloudFormation, you define what OS your instances use.

  Example: specifying an Amazon Machine Image (AMI) for an EC2 instance to use Amazon Linux 2023.

So…Types and Examples of OSes?

1. **Desktop / Laptop Operating Systems:**

i. Windows (by Microsoft) → most common desktop OS.

ii. macOS (by Apple) → used on MacBooks and iMacs.

iii. Linux distributions – open source and free:

- Ubuntu
- Fedora
- Debian
- Arch Linux
- CentOS
- Red Hat Enterprise Linux (RHEL)

2. **Mobile Operating Systems:**
- Android (built on Linux kernel; by Google)
- iOS (by Apple)
- HarmonyOS (by Huawei)



3. Server Operating Systems:

- Ubuntu Server
- CentOS / Rocky Linux / AlmaLinux
- Red Hat Enterprise Linux (RHEL)
- Windows Server

4. **Cloud & Specialized Operating Systems:**

- Amazon Linux (used on AWS EC2)
- Google Container-Optimized OS
- CoreOS / Flatcar Linux (container-focused)
- Oracle Linux

5. Embedded / IoT Operating Systems:

- Raspberry Pi OS
- Android Things
- FreeRTOS (used in microcontrollers)

The key takeaway is that, whether you're on your phone, your laptop, or a cloud server, there's always an operating system behind it, managing the hardware and allowing your applications to run.

Focusing on Linux: What Is Linux…Again?

Linux is an open-source operating system originally created by Linus Torvalds in 1991.

It is free, secure, and used almost everywhere; servers, Android phones, routers, supercomputers, and cloud infrastructure.

Some Terms and What they Mean

- Linux : A type of operating system
- Linux Distributions (distros): A full OS package using the Linux kernel (e.g., Ubuntu, Fedora, Debian, CentOS, Red Hat Enterprise Linux, Kali etc)
- GUI: The graphical interface with windows, icons, mouse
- CLI / Terminal: Text-based interface where you type commands. PS: If you can't navigate a terminal, you can't work in cloud. This isn't optional knowledge.
- Shell: The program running in the terminal that processes commands (bash, zsh, ksh, fish, powershell)
- Kernel: The core part of Linux or an OS that talks to hardware.

Think of the kernel as the engine of a car, and the shell as the dashboard and steering wheel that you use to control it.

Navigating the File System: Core Linux Commands

Pwd: present working directory	Show current directory
ls	List files
cd	Change directory
Mkdir: make directory	Create directory
touch	Create empty file
rm	Remove file
mv	Move or rename file
cp	Copy file
clear	Clear screen
cat	View file content

File Operation Commands

- cp file1 file2 # copy file
- mv file1 folder/ # move or rename
- cat file.txt # show file content
- less file.txt # scroll through file
- head -n 5 file.txt # show first 5 lines
- tail -n 5 file.txt # show last 5 lines

*** Linux doesn't always have GUIs, so you can edit text files directly.**
Common editors: nano, vi/vim, gedit.

- nano file.txt # type and save with Ctrl+O, exit with Ctrl+X

- vi file.txt # press 'i' to insert, 'Esc' to exit insert, ':wq' to save and quit

Seeking Help

man (manual pages) shows help for commands: man ls, man mkdir

--help option: ls, --help

User, Groups & Permissions



What is a User in Linux?

In Linux, A user is an account that allows a person or system service to interact with the Linux system. User information is usually stored in /etc/passwd. **To know your user in Linux, try whoami.**

Each user has: a Username, UID (User ID), Home directory, Default shell

In a company, users typically represent employees or services.

Why are there system users? Linux creates special users for system services like: systemd-network, www-data, mysql, daemon. This improves security, because each service runs with limited permissions.

What is a Group in Linux?

A group is a collection of users used to manage permissions easily. Instead of giving permissions to many users individually, Linux assigns permissions to a group.

Purpose of groups

- **To Simplify Permissions Management:** Instead of assigning permissions to each user individually, you can assign permissions to a group, and all users in that group inherit those permissions.
- **To Organize Users:** Users with similar roles or responsibilities can be grouped together. For example, all developers might be in the developers group, while system administrators are in the admin group.
- **To Control Access to Files and Directories:** By assigning a file to a group, you can control who among the group can read, write, or execute that file. Linux files and directories have permissions for: **Owner (user), Group, and Others**.

How do Groups Work?

Each user belongs to at least **one primary group** (defined in /etc/passwd) and can belong to **multiple secondary groups** (defined in /etc/group).

Primary Group:

- One per user.
- Defined by the GID (Group Identification) in /etc/passwd.
- Controls default group ownership of new files/directories



Secondary Group(s):

- Additional group memberships for access control.
- Explicitly listed in /etc/group.
- Grant access privileges beyond the primary group, up to system limits

To view the content of a primary group: cat /etc/passwd
To view the content of a secondary group: cat /etc/group

- Example: To view a user's group;

$ **groups christopher**
= christopher : christopher developers

This means the user **christopher** has a primary group **christopher** and is also part of the developers group.

Managing Groups and Users–Common Commands

1. sudo adduser john: adduser is easier because it automatically creates the home directory and asks for a password.

2. sudo useradd john: This creates the user without a home directory or password. To create the home directory and set a passwd, use, **sudo useradd -m john and sudo passwd john.**

3. groupadd cloud: Creates a new group named cloud. Used to organize users with similar permissions (e.g., cloud team).

4. usermod -aG developers alice: Adds user alice to the secondary group devs, without removing her from any existing groups. Safe way to add a user to a new group without overwriting their current group memberships.

- -a = append (only works with -G)
- -G = list of supplementary groups

5. usermod -G developers alice (⚠️ Without -a): Sets only devs as alice's secondary group.

It removes all other secondary groups. This can unintentionally wipe out important group memberships, such as sudo, for the user.

6. groups alice: Lists all groups that alice belongs to (primary + secondary). User-friendly and quick.

7. id alice: Shows UID (User ID), GID (Primary Group ID), and All groups (names + numeric IDs). More detailed than groups.

8. gpasswd -d alice devs: Removes alice from the devs group. Cleanly deletes a user from a secondary group without touching others.

9. userdel / groupdel : For deleting users or groups. PS: **Use && for multiple commands**

A sample human user account separated in 7 fields by 6 colons– alice:x:1001:100:Alice:/home/alice:/bin/bash
lilian:x:1003:1003:Lilian,,,:/home/lilian:/bin/bash

Field Number	Field Value	Description
1	alice	Username – The login name of the user.
2	x	Password Placeholder – Means the password is stored in /etc/shadow
3	1001	User ID (UID) – A unique number identifying the user.
4	100	Group ID (GID) – The primary group ID for the user.
5	Alice	GECOS field – Optional comment, usually contains full name or description.
6	/home/alice	Home Directory – The user's default working directory after login. Where the user's personal files live.
7	/bin/bash	Login Shell – The shell program the user gets when they log in (e.g., Bash).

A sample system user account separated in 7 fields by colons– systemd-network:x:102:103:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin

Field Number	Field Value	Description
1	systemd-network	Username – Used internally by systemd for network services.
2	x	Password Placeholder (actual pswds are saved in /etc/shawdow)
3	102	User ID (UID) – System accounts typically have UIDs below 1000.
4	103	Group ID (GID) – Primary group for this system service.
5	systemd Network Management,,,	GECOS field – Optional comment, usually contains full name or description. Human-readable description.
6	/run/systemd	Home Directory – Not a real home directory; system services usually use /run.
7	/usr/sbin/nologin	Like /bin/false. This is intentional for security—services should not have shells.

Other Linux user-management commands

1. **whoami:** Prints the current username you're logged in as. (Another name for root is the superuser)

Why you'd use it: To confirm which user you're operating as — helpful when using sudo, su, or switching between users.

2. **id:** Shows user ID (UID), primary group ID (GID), and all group memberships for the current user.

Why you'd use it: To check your user privileges or verify if you belong to certain groups (e.g., sudo, docker, developers).

3. **adduser alice:** Creates a home directory: /home/alice

- Adds a user entry to /etc/passwd
- Assigns a UID and GID
- Adds default shell, home, and group
- Prompts for user details



Why you'd use it: To create a new local Linux user — e.g., for a new employee, team member, or system task account.

4. **passwd alice:** Allows you to set or change the password for user alice.

Why you'd use it: To let alice log in securely, every user must have a password (unless they're disabled or use SSH keys).

5. **su - alice:** Switches the current user to alice and starts a new login shell (- means "login as if from scratch").

Why you'd use it: To test alice's environment or become alice to perform specific tasks under that user's permissions.

File Permissions

Linux uses a **permission-based model** to control access to files and directories. The main building blocks are:

1. **Permissions: r (read), w(write), x(execute). To view permissions to a file, try ls -l filename**

Permission/Symbol	Meaning on Files	Meaning on Directories/Folder
r	Can view file content	Can list directory contents (ls)
w	Can modify or delete the file	Can add or remove files inside the directory
x	Can run the file (if it's a script or binary)	Can enter (cd) into the directory

2. **User Types: Owner, Group, Others:** Each file or directory has 3 sets of permissions

- Owner: The user who owns the file (often the creator)
- Group: Users who are members of the file's group
- Others: Everyone else (not the owner or in the group)

Changing File Permissions: chmod

Syntax for changing permissions: chmod [who][+/-=][permission] filename (where r=4, w=2, x=1, 0 = no access at all, u= user/owner, g = group, o = others, a = all, + = add, - = remove, = = only)

- **chmod u+x script.sh** # Add execute to user (owner)
- **chmod g-w report.txt** # Remove write from group
- **chmod o=r file.txt** # Set others to read-only
- **chmod a+x run.sh** # Add execute to all (user, group, others)

Chmod o+w birthday_wish.txt
r-- 4,0,0

Hint: When you type ls -l filename, the output is explained thus: [file type][owner permissions][group permissions][others permissions]
Where: 7 = rwx (4+2+1), 6 = rw- (4+2), 5 = r-x (4+1), 0 = ---

To change file ownership:

- chown new_owner file
- chown new_owner:new_group file

Examples:
Change only the owner: c document.txt
Change owner and group: chown bob:devs script.sh eg. chown peter:infra index.ts
Change only the group: chown :marketing report.pdf

Practice Example
touch test.txt
ls -l test.txt
chmod 640 test.txt
ls -l test.txt
chown alice:devs test.txt
ls -l test.txt



Linux Process Management

First; What are processes in linux os?

Processes are the fundamental units of execution. They represent running programs or tasks and manage resources such as memory, CPU time, and input/output operations. When you execute a command or run a program, a process is created to handle its execution.

Types of Processes in a Linux OS

Parent & Child: In Linux/Unix, processes are born from other processes. When one process creates another, the creator is the Parent, and the new one is the Child. This creates a tree structure (you can see this with the pstree command).

Daemon: These are processes that have "detached" from their parent and their terminal. They don't have a parent "watching" them, so they run silently in the background, usually waiting for a system event or a request.

Orphaned: If a Parent process crashes or terminates while the Child is still running, the Child becomes an Orphan. In a modern system, these orphans are immediately adopted by systemd (or init, the PID 1 process), which makes sure they don't get lost in the void.

Zombie: A Zombie is a process that has finished execution but is still in the system's "process table." It is dead, but its parent hasn't "read" its exit status yet. It uses no CPU and no memory, but it is still consuming a slot in the system's process table. If you have too many, the system can't spawn new processes.

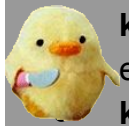
Process States

Linux processes can be in one of several states during their lifecycle and a process' state is about its activity or progress in time.:

- **New:** The process is currently being created.
- **Ready:** It is loaded into memory and waiting for its turn to get CPU time. It's essentially "standing in line."
- **Running:** The CPU is actually executing the instructions of the process.
- **Sleeping (S):** The process is waiting for an event (such as I/O operations or user input).
- **Blocked / Waiting:** The process cannot continue because it's waiting for an event (like a hard drive reading a file or a network packet arriving). It voluntarily gives up the CPU until the event occurs.
- **Terminated:** The process has finished its execution and is being cleaned up by the OS.

Some Process Management Commands

- **ps:** Displays information about running processes, including their PID, status, and other details. Example: ps aux.
- **top:** Provides a real-time, dynamic view of system processes, resource usage, and performance.
- **htop:** A more user-friendly, interactive version of top, allowing for better process management and filtering.



kill: Sends a signal to a process, usually to terminate it gracefully and for cleaning Up Zombie or Orphaned Processes.. For example, kill <PID> sends a SIGTERM signal to gracefully stop a process.
killall: Kills all processes with a given name. Example: killall firefox terminates all Firefox processes.

- **bg:** Resumes a suspended job in the background.
- **fg:** Brings a background job to the foreground.
- **Sleep:** E.g **sleep 100 &** sleeps the shell for 100 secs (& means to run the command in the background): Used for adding a delay between commands, to prevent a script from running too fast or overwhelming a resource, and useful in automation scripts where you retry a failed operation after a pause, prevent flooding a service or server with too many requests. Etc

Linux File System, Virtual Storage, Mounting Storage

Basically, Linux handles filesystems, mounts storage devices, and uses virtual memory systems like tmpfs (temporary file storage), and the Linux file system is like a tree, with a single root /. Everything lives under it…files, directories, and devices.

Important Linux folders and their typical responsibilities

S/N	Folder	Responsibility	Example	Analogy
1.	/ - Root Directory	Manages the entire file system hierarchy.	/	Main house
2.	/bin – Essential binaries	Contains basic executable programs needed by all users.	ls, cat, cp, mv	Kitchen tools (everyone can use)
3.	/sbin – System binaries	Executables for system administration (mostly for root).	ifconfig, reboot, fdisk	Maintenance tools (only admin uses)
4.	/etc – Configuration files	Stores system-wide configuration files.	/etc/passwd (user accounts), /etc/fstab (mount points)	House rules & instructions
5.	/home – User directories	Each user has a personal folder here.	/home/mayowa	Bedrooms (personal space)
6.	/var – Variable data	Stores files that change frequently.	logs (/var/log/), spool files, caches	Fridge (stuff changes constantly)
7.	/tmp – Temporary files	For short-lived files used by applications or the system.	Usually cleared on reboot	Trash bin (temp. storage)
8.	/usr – User programs & data	Contains installed programs and libraries for users.	/usr/bin → user binaries /usr/lib → libraries /usr/share → shared files (docs, icons)	
9.	/dev – Device files	Represents hardware devices as files.	/dev/sda (disk), /dev/tty (terminals)	

Package Management

Objective: Learn How to Install, Update, Remove, and Manage Software on a Linux system using package managers.

All Linux distributions use a package manager to handle software. The commands may differ between distros, but the goal is the same: easy software management. **We will be using apt to install NGINX today.**

Tool	System	Use
apt/dpkg	Debian/Ubuntu	Most common
yum/dnf	RedHat/CentOS/Fedora	Used in enterprise servers
winget/choco	Windows	Windows Package Managers
brew	macOS	Popular with developers on Macs

Common APT Commands

- **sudo apt update:** Update package list (Refreshes the list of available software (like refreshing an app store).
- **sudo apt install nginx:** Install a package (installs NGINX and its dependencies.)
- **apt-cache search nginx & apt-cache show nginx:** Search or view package info.
- **sudo apt remove nginx:** Removes the software but keeps configuration files.

  PS: Use **sudo apt purge nginx** to remove everything including configs.

NGINX (pronounced “engine-x”) is a high-performance web server used to deliver web content to users’ browsers. It is one of the most widely used web servers on the internet.

SSH - Secure Shell

SSH (Secure Shell) is a network protocol that allows secure remote access to a computer or server over an unsecured network (like the internet). PS: SSH is what allows you to log into a remote Linux machine from your terminal. When you create a virtual machine with VirtualBox or Vagrant, SSH is usually the protocol used to access that machine remotely.

Why is SSH used?

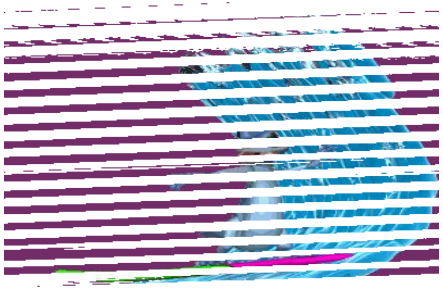
- Remote Login: Allows users/admins/devs to connect to another machine (like a server) and use it like they're physically there.
- Security: Encrypts all communication between client & server to prevent data theft or tampering.
- DevOps & Sysadmin: Used heavily for server management, troubleshooting, deployments, and automated scripts.

How does SSH Work?

1. You create a VM to ssh into. A droplet or AWS EC2 Instance.
2. Verify your key file is in the correct location
3. Start the connection by typing something like ssh user@ip address. This tells your computer… :Connect me to the machine as the user"
4. Your computer checks who the server is and once you confirm, SSH stores the server's identity so it can verify next time.
5. Once identity is confirmed, SSH creates an encrypted tunnel
6. You get a remote shell. Easy Peasy

Man Pages…The Art of Googling…Reddit…AI?

Imagine you're stuck on a live server and no one's around to help. What do you do? Professionals don't memorize every command, they just know how to search smarter and use documentation quickly.



Tips to help you SURF

- Always check documentations
- Learn how to prompt effectively
- Read your research materials
- Let AI work

What is MAN?

The man command stands for manual, it's the built-in help system for Linux and Unix-like operating systems. The man command displays manual (documentation) pages for other commands, tools, files, configuration options, and system calls.

- man ssh
- man ls
- man grep

Personal Reading….Mounting Storage

What is Mounting?

In Linux, devices (USBs, hard disks, etc.) don't appear in the filesystem until they're mounted, which means attached to a directory like /mnt, /media, or /home. Linux treats everything like a file, even disks and partitions. Mounting is essential because Linux doesn't have drive letters (like Windows does with C:, D:, etc.). Instead, it uses a single directory tree, with all files and devices integrated into one unified structure, often starting from the root directory /

Why would a system admin want to mount a disk (fake) or use tmpfs?

- To isolate logs or temporary files
- To sandbox processes.
- To test storage behavior before using real disks

Now, try this task. Here's how I want you to imagine what we're doing with mounting in Linux, using the example of a USB flash drive for this task.

- When we create a file called disk.img, think of it as a blank USB drive. It's empty space, nothing useful yet, but we're going to treat it like a real disk.
- Next, we run mkfs.ext4 on that file. That's like formatting the USB stick, we're preparing it so Linux can understand how to store files on it. Just like when you format a flash drive before use.
- Then we create a directory like /mnt/fakedisk. Think of that as plugging your USB into your computer. You need a place for the system to access the disk, this directory becomes the "plug."
- Now, we use the mount command to attach the fake disk to that directory. It's like when the USB drive appears in your File Explorer: you can open it, browse files, save stuff.
- Finally, when we run umount, we're safely ejecting that USB. The system detaches it so it's no longer in use, just like hitting "Eject" before physically removing a flash drive.

So even though we're working with a fake file, we're learning how Linux treats real disks, whether it's a USB, a hard drive, or even cloud storage. The commands are the same.

Task: Create a fake disk file, format it to like a look real disk, and mount it to a directory, so it behaves like an actual storage device.

Step 1: Create a fake disk file (100MB) fallocate -l 100M disk.img

- This creates a file called disk.img that is 100 megabytes in size.
- It's just a blank file for now, but we'll treat it like a disk.
- Think of it as a "virtual hard drive."

Step 2: Format the fake disk with a filesystem mkfs.ext4 disk.img

- This formats disk.img with the ext4 filesystem, just like you would a USB or hard drive.
- Now Linux can store files inside this fake disk.

Step 3: Create a mount point (a folder) mkdir /mnt/fakedisk

- We need a directory to mount the fake disk onto. This is where we'll "see" and interact with the contents of the disk.

Step 4: Mount the fake disk sudo mount disk.img /mnt/fakedisk

- This action here attaches the disk.img to /mnt/fakedisk.
- Now if we run: ls /mnt/fakedisk. We'd see it acts like a real empty drive
- Now, we cd into /mnt/fakedisk and create files

Step 5: Verify df -h /mnt/fakedisk, mount | grep fakedisk

- This df -h shows storage info so we can see our 100MB disk.
- And mount confirms it's mounted.

Step 6: Unmounting sudo umount /mnt/fakedisk

- This detaches the fake disk.
- Now /mnt/fakedisk is just an empty folder again.

Practice Questions

1. What command would you use to see all files in a directory, including hidden ones?
2. A file has permissions `-rw-r--r--`. Who can write to it?
3. How do you switch to the root user in the terminal?
4. What command installs a package on an Ubuntu system?
5. You notice a process is consuming too much CPU. What command shows you running processes?

Extra learning materials

- https://linuxize.com/post/how-to-create-users-in-linux-using-the-useradd-command/
- https://linuxize.com/post/how-to-delete-group-in-linux/
- https://www.w3resource.com/linux-system-administration/groups.php
- https://linuxize.com/post/chmod-command-in-linux/
- https://www.geeksforgeeks.org/linux-unix/linux-commands-cheat-sheet/
- https://linux-commands.labex.io/
- https://overthewire.org/wargames/bandit/
-

